Transcender: Cross-Layer Composition Control for Adaptive Transformer Inference

Beomjune Son

March 2026

Abstract

Adaptive-depth inference is increasingly relevant for on-device transformer deployment, yet its practical limits on consumer hardware remain undercharacterized. We argue that the binding constraint is not exit-point selection alone, but how intermediate-depth and full-depth signals are composed without corrupting the output distribution. We present Transcender, a cross-layer composition control framework for adaptive transformer inference, and evaluate this framing across three empirical tracks on a single Apple M1 Pro system using the MLX runtime, producing a mixed but coherent result. On two sparse MoE architectures (Track A), `top1_agree` composition at the penultimate layer with entropy-gated exit achieves 0.870 exact match on GPT-OSS 20B (47 of 63 prompts perfect) and 0.837 on Qwen3-30B-A3B (36 of 63 perfect), each with a sharp quality cliff one layer earlier. Track B provides a scoped negative comparison: a matched-scope naive Gemma 3 4B-IT to GPT-OSS 20B cross-model cascade reaches only 0.021 exact match at 0.18 tokens per second (TPS) on the local Apple MLX runtime. Track C extends the evaluation to dense architectures. On Gemma 3 4B-IT, compute-both agreement-aware composition recovers 0.807 exact match at both L31 and L20, versus 0.198 and 0.026 for fixed exit at those layers, while real selective-depth remains practically weak. Focused late-checkpoint follow-up on Llama 3.1 8B and Mistral 7B reproduces the same pattern: fixed exit fails materially, compute-both `top1_agree` recovers full-depth output, and real selective-depth still trails the matched full-depth baseline. The resulting picture is architecture-dependent: a viable penultimate-layer selective-exit frontier on both tested sparse MoE families, but no practical selective-depth frontier on the tested dense architectures.

1 Introduction

Variable-compute inference for transformer language models is commonly framed in one of two ways: early exit within a single model [2], or draft-verifier decoding across models [6]. In both cases, the central question is typically *when* the model has done enough work to emit the next token. We argue that this framing is incomplete.

This question has particular practical relevance for consumer-hardware deployment, where transformer models run on devices with constrained unified memory, batch size is typically one, and inference latency directly determines user experience. Adaptive-depth mechanisms that reduce per-token compute without a secondary model or server-side orchestration are of direct interest in such settings. All experiments in this work were conducted on a single Apple M1 Pro system with 32 GB unified memory using the Apple MLX runtime.

We argue that the binding constraint in the tested setting is not exit-point selection alone, but *cross-layer composition control*: when an intermediate layer and the final layer disagree, a system needs a principled rule for deciding whether to trust the shallow path, the deep path, or a combination of the two. Without such a rule, adaptive-depth inference either collapses into

low-quality fixed exit or pays the full cost of computing both paths without recovering a practical speed-quality tradeoff.

We present evidence supporting this reframing across three tracks. Track A identifies a same-model adaptive-depth operating point on GPT-OSS 20B, with cross-model reproduction on Qwen3-30B-A3B. Track B demonstrates that a naive cross-model cascade can fail severely when tokenizer mismatch, distribution mismatch, and orchestration overhead compound. Track C characterizes both the promise and the current limits of composition control on dense models: compute-both agreement-aware composition recovers substantial quality on Gemma 3 4B-IT, Llama 3.1 8B, and Mistral 7B, but real selective-depth implementations do not yet beat their matched full-depth baselines on this runtime.

2 Contributions

1. **Consumer-hardware adaptive-depth characterization.** We provide a multi-family empirical characterization of adaptive-depth inference on a consumer Apple Silicon system (M1 Pro, 32 GB unified memory), reporting both quality and runtime measurements across five models and two architecture classes under the Apple MLX runtime.

2. **Cross-layer subspace mismatch.** We measure that hidden states from different transformer layers occupy geometrically separated subspaces, indicating that naive hidden-state interpolation is empirically unsupported as a composition mechanism. Measured separation ratios reach 4.11× on GPT-OSS 20B and 2.46× on Gemma 3 4B-IT.

3. **Agreement-aware logit-space composition.** We propose a conservative composition rule that blends logits only when the shallow and deep paths agree on token identity. This outperforms both fixed exit and naive blending across the tested MoE and dense architectures.

4. **Empirical characterization of the cascade tax.** We measure the compounding cost of tokenizer mismatch, distribution mismatch, and two-model orchestration in a naive cross-model cascade, providing a concrete negative baseline.

5. **Family-sensitive dense depth profiles.** We observe that KL-divergence depth profiles differ across dense model families: Gemma exhibits a late plateau (L20–L29), Llama a milder middle plateau (L12–L14), and Mistral an earlier plateau (L2–L8). The benchmark-side limitation generalizes more cleanly than the internal profile geometry.

6. **Real dense selective-depth speed validation.** We implement continuation-safe selective-depth runtimes on three dense model families. All physically skip deep layers on some decode tokens, but none recovers a practical speed-quality frontier on this runtime.

3 Related Work

Adaptive-depth inference for transformers has been studied along several axes. Elbayad et al. [2] introduce per-token depth allocation via learned halting classifiers. CALM [8] extends this with calibrated confidence thresholds, demonstrating early exit on encoder-decoder models with a learned consistency mechanism. DeeBERT [9] applies early exit to BERT classification tasks using entropy-based halting. These approaches focus on *when* to exit; the present work shifts emphasis to *how* to compose intermediate and full-depth signals when the exit decision is uncertain.

Speculative decoding [6, 1] addresses a related problem from the opposite direction: a small draft model proposes tokens, and a larger verifier accepts or rejects them. LayerSkip [3] combines early exit with self-speculative decoding within a single model, using early layers as the draft and later layers as the verifier. Track B's cascade result is a scoped negative baseline for a naive cross-model configuration, not a comparison to production speculative decoding systems.

Mixture-of-Depths [7] trains transformers with per-token depth routing, allowing tokens to skip entire layers during forward passes. This is structurally related to Track A's entropy-gated exit but differs in requiring training-time routing decisions rather than post-hoc inference-time composition. Switch Transformers [4] demonstrate that sparse expert routing enables efficient scaling; the MoE structure in Track A's tested models creates natural per-token compute variation that may interact with adaptive-depth mechanisms differently than dense architectures, a distinction that Track C's results empirically support.

The present work differs from the above primarily in emphasis. Rather than proposing a new exit criterion or training procedure, we characterize the empirical operating limits of composition control across architecture classes and report both positive and negative results on consumer hardware.

4 Method

4.1 Cross-Layer Subspace Mismatch

Let h_i and h_j denote hidden states from layers i and j with $i < j$. When projected into a shared PCA space, we measure cross-layer separation using

$$\text{SeparationRatio}(i, j) = \frac{\|c_i - c_j\|_2}{\max(\sigma_i, \sigma_j)}, \tag{1}$$

where c_i and c_j are cluster centroids and σ_i, σ_j are within-cluster standard deviations. Ratios exceeding 2× suggest that hidden states from different layers are geometrically misaligned, making naive interpolation between them empirically unstable. We observe 4.11× on GPT-OSS 20B and 2.46× on Gemma 3 4B-IT under our setup.

4.2 Agreement-Aware Logit Composition

Given the observed cross-layer subspace mismatch, Transcender avoids hidden-state blending and instead projects intermediate and final representations into a shared logit space for composition. Let ℓ_t^e denote the logits at exit layer e for token step t, and ℓ_t^N the logits at full depth N. The composition rule is

$$\ell_t^{\text{out}} = \begin{cases} (1-\alpha)\,\ell_t^N + \alpha\,\ell_t^e, & \text{if } \arg\max \ell_t^e = \arg\max \ell_t^N, \\ \ell_t^N, & \text{otherwise.} \end{cases} \tag{2}$$

We refer to this as the `top1_agree` rule. Its design is conservative: logits are blended only when the shallow and deep paths already agree on token identity; otherwise, the shallow signal is discarded entirely.

4.3 Selective-Depth Runtime

The dense selective-depth runtime used in Track C differs from the compute-both composition path. Prompt prefill always runs at full depth to ensure deep-cache validity at decode start.

During autoregressive decoding, layers $0 \dots e$ execute first, a confidence score $s(\ell_t^e)$ is computed, and the early token is accepted only when confidence exceeds a threshold:

$$a_t = \mathbf{1}[s(\ell_t^e) \geq \tau], \qquad \hat{y}_t = \begin{cases} \arg\max \ell_t^e, & a_t = 1, \\ \arg\max \ell_t^N, & a_t = 0. \end{cases} \tag{3}$$

When continuation is required after one or more accepted shallow tokens, the previously skipped tokens are replayed through the deep layers to restore cache correctness. This replay cost is included in all reported runtimes and is essential to the interpretation of the results.

5 Experimental Setup

All results were collected on a single Apple M1 Pro system with 32 GB unified memory using the Apple MLX runtime. The Track A evaluation uses a suite of 64 prompts spanning six categories: expository (13), code/technical (10), reasoning/logic (10), creative/open-ended (10), short-answer factual (11), and list/structured output (10). Each prompt generates 48 tokens under greedy decoding. Aggregate metrics exclude the first prompt as warmup to reduce distortion from JIT compilation and initial memory allocation, yielding N=63 scored prompts. Track A remains reported on that canonical N=63 denominator. When the paper makes direct cross-track comparisons, it uses the shared expository subset defined by canonical prompt IDs $P1$–$P5$, with $P1$ excluded as warmup so that $P2$–$P5$ are scored. We treat those matched-scope tables as supporting comparisons rather than as replacements for the main Track A table.

The primary quality metric is exact token-sequence match against a full-depth baseline on the same model. Secondary metrics include generation throughput (tokens per second), time to first token (TTFT), elapsed time, peak memory, and layer-saving statistics where applicable.

The evaluated models are: GPT-OSS 20B, a 24-layer Mixture-of-Experts model (32 experts, top-4) in MXFP4 quantization; Qwen3-30B-A3B, a 48-layer MoE model (128 experts, top-8) in 4-bit quantization; Gemma 3 4B-IT [5], a 34-layer dense model; and two MLX 4-bit dense models—Llama 3.1 8B Instruct and Mistral 7B Instruct v0.3. Track A evaluates both MoE models. Track B combines Gemma 3 4B-IT and GPT-OSS 20B in a naive cross-model cascade. Track C evaluates Gemma 3 4B-IT across six adaptive modes and dedicated selective-depth runtimes, then extends to late-checkpoint validation on Llama and Mistral.

6 Results

6.1 Track A: Same-Model Adaptive Depth

Track A is the primary empirical result. As shown in table 1, `top1_agree` at Layer 22 achieves 0.870 exact match across 63 scored prompts (47 of 63 perfect), with an average of 0.49 layers saved per generated token via entropy-gated exit. At L22 on a 24-layer model, each early-exiting token physically skips one MoE-MLP block (layer 23); the 0.49 figure therefore reflects that approximately 49% of decode tokens triggered the entropy gate and exited early. L22 is the only viable operating point in the local frontier: L21 drops to 0.703 exact match (27 of 63 perfect), and L20 to 0.426 (2 of 63 perfect). The layer savings are not a compute-both artifact—they reflect genuine adaptive compute reduction in the MLX runtime.

Table 1: Track A local exit-layer frontier on GPT-OSS 20B (N=63 scored prompts). The `avg_layers_saved` column reports the mean number of layers physically skipped per generated token via entropy-gated exit. On a 24-layer model at exit layer e, each early-exiting token skips $24 - (e+1)$ layers.

Configuration	Gen TPS	Exact Match	Avg. Layers Saved
Full depth (L23)	21.9	1.000	0.000
L22 `top1_agree`	18.6	0.870	0.490
L21 `top1_agree`	19.4	0.703	1.130
L20 `top1_agree`	18.0	0.426	1.280

Table 2: Cross-model MoE validation on Qwen3-30B-A3B (N=63 scored prompts). On a 48-layer model at exit layer e, each early-exiting token skips $48 - (e+1)$ layers.

Configuration	Gen TPS	Exact Match	Avg. Layers Saved
Full depth (L47)	37.3	1.000	0.000
L46 `top1_agree`	32.1	0.837	0.760
L45 `top1_agree`	32.7	0.463	1.535

6.1.1 Cross-Model MoE Reproduction

To test whether the MoE frontier generalizes beyond GPT-OSS 20B, we ran the same benchmark on Qwen3-30B-A3B, a structurally different sparse MoE model (48 layers, 128 experts, top-8 routing). As shown in table 2, `top1_agree` at Layer 46 (penultimate) achieves 0.837 exact match (36 of 63 perfect) with 0.760 average layers saved per token. Quality drops sharply at L45 (0.463 exact match, 6 of 63 perfect), confirming a single viable penultimate-layer operating point with the same structural pattern as GPT-OSS. The Qwen3 frontier is weaker than GPT-OSS (0.837 vs. 0.870), indicating that operating-point quality is model-dependent, but the frontier structure—a viable penultimate exit with a sharp cliff one layer earlier—reproduces across both tested sparse MoE families without any threshold tuning.

6.1.2 Category-Level Analysis

The N=63 prompt suite spans six task categories. The quality cliff at the penultimate-layer frontier is sharp on both models (figure 1), and penultimate-layer quality is task-sensitive (figure 2). GPT-OSS L22 achieves perfect output on all 10 creative/open-ended prompts (1.000 EM) and near-perfect on list/structured output (0.977 EM, 9/10 perfect), but drops to 0.776 EM on expository prompts (8/12 perfect) and 0.765 EM on code/technical prompts (7/10 perfect). Qwen3 L46 shows a similar pattern: reasoning/logic is strongest (0.950 EM, 7/10 perfect) while expository is weakest (0.656 EM, 4/12 perfect).

Failure analysis reveals architecture-dependent divergence patterns. Among GPT-OSS's 16 penultimate-layer failures, divergence positions are distributed across early (tokens 1–10: 6), mid (11–24: 6), and late (25+: 4) regions. Qwen3's 27 failures skew late: 15 of 27 (56%) diverge after token 25, suggesting that Qwen3's composition errors tend to accumulate gradually rather than appearing at generation onset. On GPT-OSS, all four expository failures diverge within the first 10 tokens, consistent with the hypothesis that sustained coherent generation taxes the shallow path most heavily. Qwen3's eight expository failures are more distributed (4 early, 1 mid, 3 late), suggesting that the relationship between task type and divergence timing is itself architecture-

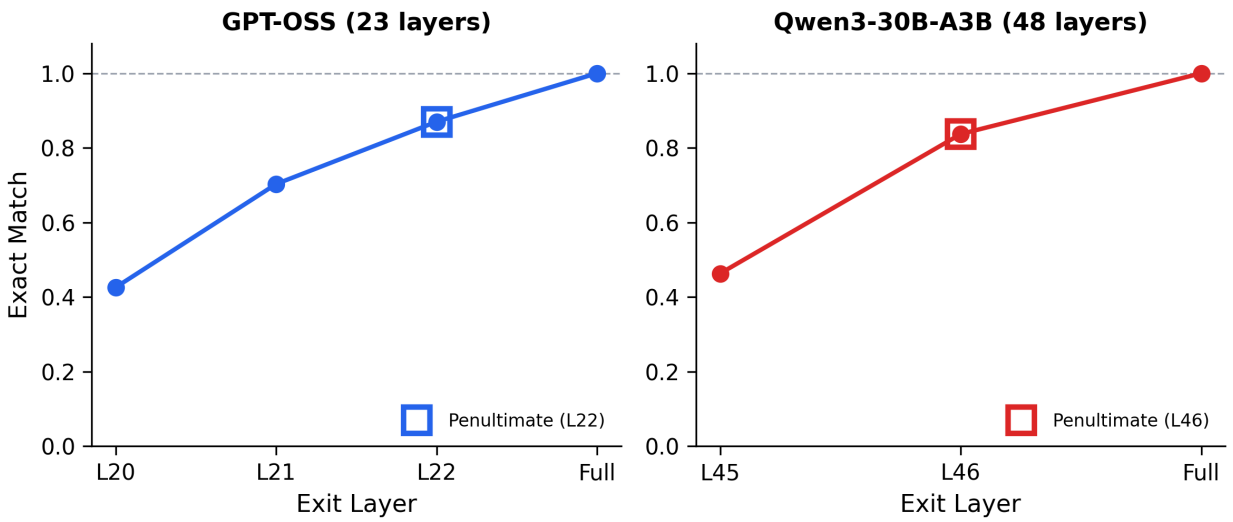

Figure 1: Quality cliff at the penultimate-layer frontier. Both models show a sharp drop from penultimate to penultimate−1 layer, but Qwen3's cliff is steeper (0.837→0.463 vs. 0.870→0.703).

Table 3: Track B comparison on the local Apple MLX runtime. The Track A row is recomputed on the same shared expository subset ($P2$–$P5$ scored after warmup) for matched-scope context.

Mode	Gen TPS	Exact Match	Peak Mem (GB)	TTFT (s)
Draft only (Gemma 3 4B-IT)	10.35	0.006	7.28	0.66
GPT-OSS full depth	25.76	1.000	12.96	1.58
Naive cascade	0.18	0.021	20.19	4.34
Track A L22 `top1_agree` (matched subset)	23.76	1.000	12.96	0.83

dependent.

6.2 Track B: Cross-Model Cascade Baseline

Track B is not a claim about cascade systems in general. It is a scoped negative baseline for this particular naive implementation, this Gemma 3 4B-IT / GPT-OSS 20B model pair, and this local runtime. As shown in table 3, the matched-scope rerun still reaches only 0.021 exact match at 0.18 TPS with 20.19 GB peak memory. On the same matched expository subset, the canonical Track A L22 policy reaches 1.000 exact match at 23.76 TPS, so the cascade gap is not an artifact of mixing denominators. The primary causes are tokenizer mismatch, output-distribution mismatch, and two-model orchestration overhead. We treat this result as evidence about the cascade tax in this specific configuration, not as a general statement against speculative decoding [6].

6.3 Track C: Dense Model Evaluation

Track C establishes that fixed exit is catastrophic on the tested dense model. Even the latest fixed exit at L31 saves only 6% of layers while falling to 0.198 exact match. By contrast, compute-both agreement-aware composition at L31 reaches 0.807 exact match (table 4), well above both fixed exit and naive blending (+11.98 percentage points over naive blend). This matched-scope Gemma table is the paper's main dense-model boundary comparison against Track A and Track B; the

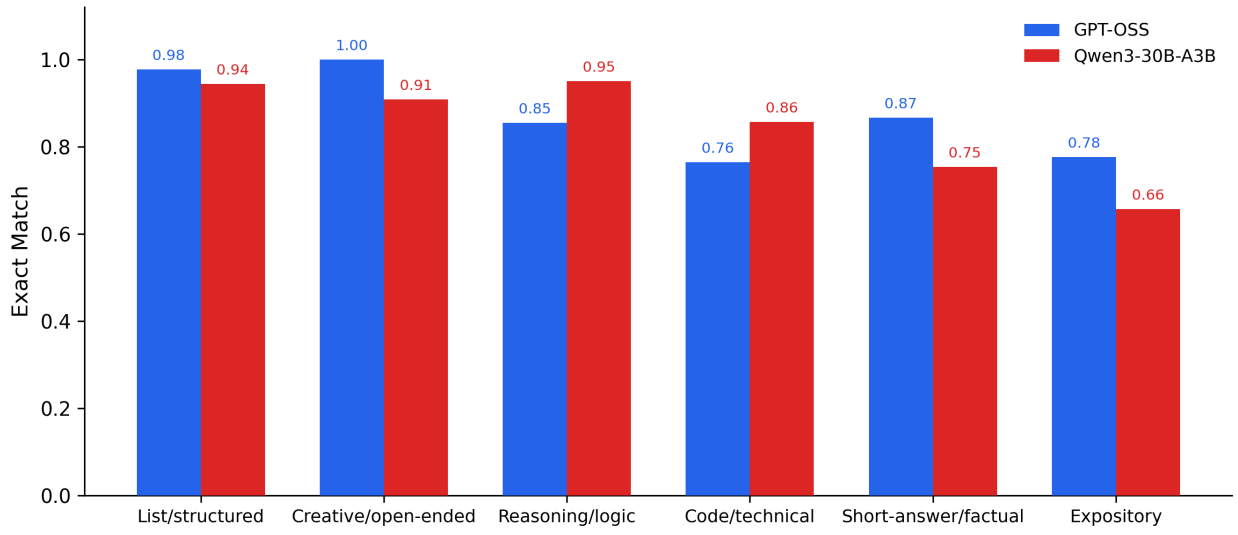

Figure 2: Penultimate-layer exact match by prompt category. Both models struggle most with expository prompts requiring sustained coherent generation, while structured and creative categories are most robust.

Table 4: Track C adaptive benchmark on Gemma 3 4B-IT, reported on the shared expository subset ($P2$–$P5$ scored after warmup).

Mode	Gen TPS	Exact Match	Layers Saved	Agreement Rate
Full depth L33	20.46	1.000	0%	—
Fixed exit L16	29.16	0.010	50%	—
Fixed exit L20	25.74	0.026	38%	—
Fixed exit L31	18.52	0.198	6%	—
`top1_agree` L31	14.49	0.807	0%	81.9%
Naive blend L31	14.10	0.688	0%	100%

larger point is qualitative rather than pooled-statistical. It shows that dense models can benefit from conservative composition, but not that they inherit the same practical selective-exit frontier as the sparse MoE Track A results.

6.4 Track C: Selective-Depth Speed Validation

The selective-depth extension asks whether the late-layer agreement signal at L31 can be converted into real layer skipping. The answer is affirmative operationally but negative practically. As shown in table 5, the entropy rule achieves a 34.0% realized skip rate over continuation-eligible decode steps. However, with only two of thirty-four layers available to skip, average savings remain at 2.0%. Neither selective mode exceeds the matched full-depth baseline of 19.19 TPS, and both remain near fixed-exit quality.

This distinction is important. Compute-both `top1_agree` is a quality-control result with no physical skip. Real selective depth is a speed-validation result. On Gemma 3 4B-IT at L31, replay and control overhead erased the available compute savings on this runtime.

Table 5: Selective-depth benchmark at L31 on the shared expository subset ($P2$–$P5$ scored after warmup). All numbers should be compared to the full-depth row within this table, as they come from a dedicated selective-depth runtime.

Mode	Gen TPS	Exact Match	Accept	Skip	Avg. Layers Saved
Full depth L33	19.19	1.000	—	0%	0%
Fixed exit L31	18.25	0.198	—	—	5.9%
`top1_agree` compute-both L31	14.78	0.807	81.9%	0%	0%
Selective margin L31	16.54	0.229	78.2%	3.7%	0.2%
Selective entropy L31	16.84	0.208	93.6%	34.0%	2.0%

Table 6: Selective-depth probe at L20 on the same expository subset, testing whether a larger skip budget recovers a practical frontier.

Mode	Gen TPS	Exact Match	Accept	Skip	Avg. Layers Saved
Full depth L33	15.16	1.000	—	0%	0%
Fixed exit L20	20.20	0.026	—	—	38.2%
`top1_agree` compute-both L20	10.54	0.807	28.2%	0%	0%
Selective margin L20	11.29	0.083	70.2%	13.8%	5.3%
Selective entropy L20	13.26	0.073	73.4%	41.0%	15.7%
Selective hybrid L20	11.37	0.125	61.2%	6.4%	2.4%

The L20 probe tests whether a materially larger skip budget recovers a practical frontier. Table 6 shows it does not in this configuration. The entropy rule increases average layers saved to 15.7% with a 41.0% realized skip rate, but reaches only 13.26 TPS against a matched full-depth baseline of 15.16 TPS. Quality collapses toward fixed-exit levels at 0.073 exact match. Larger skip budget alone does not resolve dense selective-depth when continuation quality degrades and replay cost remains significant.

6.5 Track C: Multi-Family Dense Follow-Up

Table 7 summarizes late-checkpoint follow-up on Llama 3.1 8B and Mistral 7B. The benchmark-side pattern from Gemma reproduces cleanly across both families: late fixed exit fails materially, compute-both `top1_agree` recovers full-depth output, and real selective-depth physically skips layers but does not recover a practical speed-quality frontier.

These results broaden the dense benchmark finding beyond Gemma, but do not justify treating Gemma's internal profile as universal. The KL-divergence depth profiles are family-sensitive: Gemma shows a late plateau at L20–L29, Llama shows a milder plateau near L12–L14 with resolution entry at L27, and Mistral shows an earlier plateau at L2–L8 with resolution entry at L26. The benchmark-side limitation therefore generalizes more cleanly than the internal profile geometry.

7 Limitations

- **Architecture sensitivity.** The strongest practical result appears on the tested MoE models. The magnitude of adaptive-depth gains is architecture-dependent, as shown by the quality difference between GPT-OSS 20B (0.870) and Qwen3-30B-A3B (0.837).

Table 7: Late-checkpoint dense-family follow-up on the shared expository subset ($P2$–$P5$ scored after warmup) on the local Apple MLX runtime.

Model	Late CP	Fixed Exit	Compute-Both	Selective	Selective TPS	Avg. Saved
Llama 3.1 8B	L29	0.151	1.000	0.469	16.60 vs 17.81	0.3%
Mistral 7B v0.3	L29	0.109	1.000	0.318	17.25 vs 29.81	0.2%

- **MoE coverage.** The cross-model sparse-MoE validation covers two families. Broader MoE generalization remains untested.
- **Track B scope.** The cascade result is specific to a naive implementation, one model pair, and one local runtime. It should not be generalized to speculative decoding as a class of methods.
- **Runtime dependence.** All throughput, latency, and memory measurements are specific to a single Apple M1 Pro system using MLX. Direct comparison to GPU-based inference is not appropriate. Off-MLX structural reproductions on an NVIDIA H200 GPU (transformers 5.3.0.dev0) confirm the penultimate-layer frontier pattern on both tested MoE families: GPT-OSS 20B gives N=63 raw-exit exact match 0.808 at L21 and 0.879 at L22 (bfloat16), closely tracking the canonical MLX L22 result of 0.870; Qwen3-30B-A3B gives 0.832 at L45 and 0.916 at L46 (float16), with the penultimate advantage holding across all 63 prompts. Cliff probes extending one layer further back confirm monotonic degradation: GPT-OSS drops to 0.589 at L20; Qwen3 drops to 0.793 at L44. The sharp quality cliff seen on MLX (Qwen3 drops from 0.837 to 0.463 at L45) is materially softer on GPU (0.916 to 0.832), confirming that frontier structure is robust off-MLX while cliff severity is runtime-sensitive. The layer ordering also survives extension from 48 to 64 generated tokens: GPT-OSS numbers change by less than 0.005; Qwen3 shows a uniform $\sim$2–3% drop at longer sequences but preserves the monotonic L44 $<$ L45 $<$ L46 structure.
- **Evaluation scale.** Track A uses 63 scored prompts across six categories, but the paper's direct cross-track comparisons still rely on a small matched expository subset with four scored prompts after warmup. This is sufficient for directional findings but insufficient for statistical confidence intervals.
- **Offline Stage B scope.** The GPU token-row analysis used for the Stage B framing is a held-out offline diagnostic based on structural reproduction runs rather than a deployment benchmark. GPU-side karma logistic classifiers achieve 0.859 precision / 0.085 error on GPT-OSS and 0.864 precision / 0.078 error on Qwen3, materially outperforming entropy-only baselines (0.564 / 0.529 precision respectively). Among tested oracle-style acceptance diagnostics (top1_agree, margin-gated, entropy-gated, two-layer cross-check), plain penultimate `top1_agree` yields the highest acceptance rate on both models without reducing composed quality. Adding entropy gating at threshold 1.5 reduces GPT-OSS acceptance rate from 87.8% to 71.0% while barely affecting Qwen3 (91.6% $\rightarrow$ 90.5%); adding margin gating or two-layer agreement also lowers acceptance without demonstrated quality benefit under the manual-reference verifier path. Oracle design is therefore model-sensitive, and more complex gating rules did not outperform single-layer top1 agreement in this configuration. These results support a framing hypothesis about penultimate-layer risk, not a production policy, a universal threshold, or a serving-speed claim.

- **Dense selective-depth.** Real selective-depth runtimes on all three tested dense families were operationally correct but did not recover a practical speed-quality frontier in this configuration.
- **Continuation mechanisms.** The current selective-depth results demonstrate that checkpoint placement and late-checkpoint confidence alone are insufficient. Better continuation criteria, token-difficulty-aware routing, and lower-overhead cache repair remain open problems.
- **Family-sensitive structure.** The benchmark-side dense limitation generalizes across tested families, but the internal KL-profile shape does not. We do not claim a single universal dense depth profile.

8 Conclusion

The evidence presented supports a narrow but consistent conclusion: adaptive-depth inference for transformers is constrained less by exit-point selection than by cross-layer composition control—how intermediate and full-depth signals are combined without corrupting the output distribution.

Track A remains the primary empirical result. On GPT-OSS 20B, `top1_agree` at the penultimate layer achieves 0.870 exact match (47 of 63 perfect), with approximately 49% of decode tokens physically skipping the final MoE-MLP block. On Qwen3-30B-A3B, the same mechanism at the penultimate layer achieves 0.837 exact match (36 of 63 perfect) with a sharp quality cliff one layer earlier. The frontier structure reproduces across both tested sparse MoE families; the operating-point quality is model-dependent. Track B provides a scoped negative baseline for this naive cascade configuration. Track C contributes three findings: fixed-exit failure on dense models, strong compute-both composition recovery, and weak real selective-depth across three tested families.

Separate offline GPU-path analysis also suggests that the penultimate acceptance question is better framed as a risk-estimation problem than as simple confidence thresholding: penultimate entropy alone is only a weak baseline, and a multi-feature offline logistic score (`karma`) is more naturally interpreted as the probability that full depth is still needed. Among tested oracle-style diagnostics, plain penultimate `top1_agree` remains the strongest simple acceptance rule; adding margin, entropy, or cross-layer gating did not improve the picture. GPU token-row triage under the manual-reference verifier path shows that $81-83\%$ of tokens match full depth at the earlier exit layer, $9-10\%$ additionally match at the penultimate layer, and $8-9\%$ require full depth. We treat that result as a cautious diagnostic extension to the main Track A/B/C evidence, not as a deployment claim or as a replacement for Track B's negative baseline and Track C's dense-model boundary evidence.

The dense-model conclusion is not that adaptive depth is impossible, but that neither fixed exit nor current selective-depth mechanisms establish a practical frontier on the tested dense architectures. The next research direction is correspondingly narrower: whether family-sensitive continuation criteria, token-difficulty-aware routing, and cache-aware continuation can close the gap. That remains to be validated.

A Offline Stage B Diagnostic Note

The GPU Track A diagnostic path also supports token-level relation-row export for an offline Stage B decision problem. This is an additive diagnostic extension to the paper's main Track A/B/C

structure, not a replacement for it. The oracle labels distinguish three cases: `earlier_correct`, `need_penultimate`, and `need_full_depth`. Stage A asks whether the earlier layer was already sufficient. Stage B starts only after that has failed and asks whether the penultimate layer is still safe to accept or whether full depth is required.

The current conclusion from this offline analysis is narrow. Simple adjacent-layer agreement is not a reliable cross-family Stage B acceptance signal, and penultimate entropy is better treated as a weak confidence-style baseline than as a full explanation of the decision. A small offline logistic model over penultimate and adjacent-relation features, summarized internally as `karma`, is better interpreted as a risk score for whether the penultimate layer remains unsafe and full depth is still needed. Lower `karma` is therefore safer. We emphasize that this remains an offline held-out result and not a production routing policy, a universal threshold, or a reason to delete the Track B and Track C framing from the paper.

B Reproduction Commands

All commands assume execution from the repository root.

Track A (GPT-OSS 20B):

```
python scripts/track_a/transcender_exit_layer_benchmark.py
```

Track A (Qwen3-30B-A3B):

```
python scripts/track_a/transcender_exit_layer_benchmark.py \
  --model-type qwen3_moe \
  --model mlx-community/Qwen3-30B-A3B-4bit
```

Track B:

```
python scripts/track_b/transcender_track_b_benchmark.py \
  --large-model /path/to/gpt-oss-20b-raw \
  --draft-model /path/to/gemma-3-4b-it \
  --prompt-suite canonical_64 \
  --prompt-ids P1,P2,P3,P4,P5 \
  --draft-chunk-tokens 16 \
  --output artifacts/track_b/\
    transcender_track_b_benchmark_matched_p1_p5_chunk16.json
```

Track C — Gemma profile and benchmark:

```
python scripts/track_c/transcender_track_c_gemma_profile.py \
  --model /path/to/gemma-3-4b-it

python scripts/track_c/transcender_track_c_gemma_benchmark.py \
  --model /path/to/gemma-3-4b-it \
  --prompt-suite canonical_64 \
  --prompt-ids P1,P2,P3,P4,P5 \
  --early-layer 16 --mid-layer 20 --late-layer 31 \
  --output artifacts/track_c/\
    transcender_track_c_gemma_results_matched_p1_p5.json
```

Track C — Gemma selective depth (matched L31 and legacy L20 probe):

```
python scripts/track_c/\
  transcender_track_c_gemma_selective_depth.py \
  --model /path/to/gemma-3-4b-it \
  --prompt-suite canonical_64 \
  --prompt-ids P1,P2,P3,P4,P5 \
  --late-layer 31 \
  --output artifacts/track_c/\
    transcender_track_c_gemma_selective_depth_results_matched_p1_p5.json

python scripts/track_c/\
  transcender_track_c_gemma_selective_depth.py \
  --model /path/to/gemma-3-4b-it --late-layer 20 \
  --margin-threshold 0.02 --entropy-threshold 0.65 \
  --include-hybrid-mode \
  --output artifacts/track_c/\
    transcender_track_c_gemma_selective_depth_L20_results.json
```

Track C — Dense family validation:

```
python scripts/track_c/\
  transcender_track_c_dense_family_validation.py \
  --model /path/to/llama-3.1-8b-instruct-4bit \
  --family llama --late-layer 29 \
  --entropy-threshold 0.15 \
  --profile-output artifacts/recon/recon_llama3_8b.json \
  --benchmark-output artifacts/dense_followup/\
    transcender_track_c_llama3_8b_results.json

python scripts/track_c/\
  transcender_track_c_dense_family_validation.py \
  --model /path/to/mistral-7b-instruct-v0.3-4bit \
  --family mistral --late-layer 29 \
  --entropy-threshold 0.15 \
  --profile-output artifacts/recon/recon_mistral7b.json \
  --benchmark-output artifacts/dense_followup/\
    transcender_track_c_mistral7b_results.json
```

GPU Track A — token-row export and offline Stage B:

```
python scripts/track_a_gpu/transcender_gpu_reproduction.py \
  --model /path/to/model \
  --emit-token-rows \
  --token-rows-output artifacts/track_a_gpu/model_token_rows.jsonl \
  --output artifacts/track_a_gpu/model_gpu_reproduction.json

python scripts/track_a_gpu/evaluate_relation_proxies.py \
  artifacts/track_a_gpu/model_token_rows.jsonl \
  --fit-karma-logistic \
  --as-json \
  > artifacts/track_a_gpu/model_stage_b_karma.json

python scripts/track_a_gpu/summarize_karma_results.py \
  --format markdown \
  artifacts/track_a_gpu/model_stage_b_karma.json
```

C Benchmark Artifacts

- `artifacts/track_a/transcender_exit_layer_benchmark_n63.json`: Track A per-prompt metrics for GPT-OSS 20B at L20, L21, L22, and L23 full-depth across 64 prompts (63 scored).
- `artifacts/track_a/qwen3_30b_a3b_exit_layer_benchmark_n63.json`: Track A per-prompt metrics for Qwen3-30B-A3B at L40, L44, L45, L46, and L47 full-depth across 64 prompts (63 scored).
- `artifacts/track_b/transcender_track_b_benchmark_matched_p1_p5_chunk16.json`: Track B matched-scope rerun on canonical prompt definitions filtered to $P1$–$P5$ ($P2$–$P5$ scored after warmup).
- `artifacts/track_b/transcender_track_b_benchmark.json`: Legacy Track B expository artifact retained for comparison with the earlier writeup.
- `artifacts/track_c/transcender_track_c_gemma_results_matched_p1_p5.json`: Matched-scope Gemma Track C benchmark on the shared expository subset.
- `artifacts/track_c/transcender_track_c_gemma_results.json`: Legacy-scope Gemma benchmark artifact retained for comparison with the earlier writeup.
- `artifacts/track_c/transcender_track_c_gemma_selective_depth_results_matched_p1_p5.json`: Matched-scope Gemma selective-depth speed validation at L31.
- `artifacts/track_c/transcender_track_c_gemma_selective_depth_L20_results.json`: Track C dedicated L20 selective-depth probe.
- `artifacts/recon/recon_llama3_8b.json`: KL profile and heuristic summary for Llama 3.1 8B Instruct 4-bit.
- `artifacts/dense_followup/transcender_track_c_llama3_8b_results.json`: Late-checkpoint validation metrics for Llama 3.1 8B.
- `artifacts/recon/recon_mistral7b.json`: KL profile and heuristic summary for Mistral 7B Instruct v0.3 4-bit.
- `artifacts/dense_followup/transcender_track_c_mistral7b_results.json`: Late-checkpoint validation metrics for Mistral 7B.
- `artifacts/track_c/gemma_kl_profile.json`: Per-layer KL divergence across the 34-layer Gemma 3 4B-IT stack.
- `artifacts/track_a/analysis/category_analysis.json`: Per-category exact match, failure lists, and divergence classification for both MoE models at the penultimate layer.
- `artifacts/track_a_gpu/gpt_oss_20b_gpu_reproduction_n63_bf16.json`: Off-MLX GPU structural reproduction of GPT-OSS 20B on NVIDIA H200 (bfloat16). N=63 raw-exit exact match 0.808 at L21 and 0.879 at L22. Diagnostic only; not a replacement for the canonical MLX Track A artifacts.
- `artifacts/track_a_gpu/qwen3_30b_a3b_gpu_reproduction_n63.json`: Off-MLX GPU structural reproduction of Qwen3-30B-A3B on NVIDIA H200 (float16). N=63 raw-exit exact match 0.832 at L45 and 0.916 at L46. Penultimate advantage confirmed; quality cliff softer than on MLX.
- `artifacts/track_a_gpu/gpt_oss_cliff_L20_L21_L22.json`, `qwen3_cliff_L44_L45_L46.json`: Cliff probe artifacts extending one layer further back (48 tokens). GPT-OSS degrades monotonically: L20=0.589, L21=0.808, L22=0.879. Qwen3: L44=0.793, L45=0.832, L46=0.916.
- `artifacts/track_a_gpu/gpt_oss_cliff_L20_L21_L22_t64.json`, `qwen3_cliff_L44_L45_L46_t64.json`: Length-robustness check at 64 tokens. GPT-OSS: L20=0.594, L21=0.807, L22=0.876 (within noise of 48-token run). Qwen3: L44=0.766, L45=0.809, L46=0.904 (uniform ∼2–3% drop, ordering preserved).
- `artifacts/track_a_gpu/gpt_oss_multi_oracle_n63.json`, `qwen3_multi_oracle_n63.json`: Multi-oracle diagnostics (top1_agree, margin-gated, entropy-gated, two-layer cross-check) for both MoE models on H200.

- `artifacts/track_a_gpu/gpt_oss_token_rows_n64.jsonl`, `qwen3_token_rows_n64.jsonl`: Per-token row data (3039 and 3072 rows) for Stage B karma logistic fitting.
- `artifacts/track_a_gpu/gpt_oss_stage_b_karma.json`, `qwen3_stage_b_karma.json`: Karma logistic fit results. GPT-OSS: precision=0.859, error=0.085. Qwen3: precision=0.864, error=0.078.

References

[1] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023.

[2] Maha Elbayad, Laurent Besacier, and Jakob Verbeek. Depth-adaptive transformer. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020.

[3] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acber, Saurabh Agarwal, Ahmed Roman, Ahmed Aly, Beidi Zhu, and Michael Bendersky. LayerSkip: Enabling early-exit inference and self-speculative decoding, 2024.

[4] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[5] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, et al. Gemma: Open models based on gemini research and technology, 2024.

[6] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[7] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter C. Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024.

[8] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[9] Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. DeeBERT: Dynamic early exiting for accelerating BERT inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.